

Chapter 3: A brief interlude

Miguel de Jesús Palacios Rivera

What makes a good abstraction? What do we want from abstractions?

We need them to hide the messy details of our system.

Coupled: when we are unable to modify module A for fear of breaking B. Locally is good but not globally.

Abstracting state aids testability

We usually try to write a simple implementation and then refactor toward a better design.

Getting decent coverage and revealing these bugs will become earlier than later very painful.

On top of that, our code is not very extensible.

Choosing the right abstractions

We need to separate each responsibility.

We need to remember the duckduckgo example from Chapter 1.

We are going to separate what we want to do from how to do it.

Implementing our chosen abstractions

How do we actually write those new tests? How do we change our implementation to make it all work?

We need to isolate the clever part of our system. **The Functional Core:** has no dependencies on external state.

Gather inputs, call our logic, apply outputs.

End-to-end or edge-to-edge testing

Testing edge to edge with fakes dependency injection

We want to test larger parts of our system and bigger chunks of the system together.

We often write tests that invoke a while system together but fake I/O.

The advantage of this approach is that our tests act on the exact same function that's used by our production code. The disadvantage is that we have to make our stateful components explicit and pass them around.

Why not just patch it out?

Mocking:

- It does nothing to improve the design.
- It tend to ve more coupled to the implementation details of the codebase.
- It leads to complicated test suites that fail to explain the code.

Difference between mocks and fakes

Mocks: are used to verify **how** something gets used.

Fakes: are working implementation of the thing they are replacing.